

Specification and Verification of Embedded and Cyber-Physical Systems

Exercise List — ESBMC 8.5

Before you start

Every command in this list was run against ESBMC **8.5.0**. Check your build first:

```
esbmc --version      # expect 8.5.0 or newer
```

Reference [3] is the tool paper; [1] describes the SMT encoding underneath it.

Three things changed since the version used in earlier editions of this tutorial, and they will show up in your terminal:

- The default solver is now **Bitwuzla**. Use `--z3`, `--cvc5` or `--boolector` to cross-check a result.
- Verification **stops at the first violated property**; the rest are listed as NOT CHECKED. Add `--multi-property` for a verdict on every property in a single run.
- Loop invariants in C are written `__ESBMC_loop_invariant(...)` and enabled with `--loop-invariant` or `--loop-invariant-check`. The Python frontend spells the same annotation `__loop_invariant(...)`.

Unless stated otherwise, the files referred to below live in `examples/` in the tutorial repository; `bmc-example.c` and `bmc-example.py` are in `exercises/`, next to this handout. `examples/README.md` records the command and the expected verdict for each example, and `examples/run-all.sh` re-checks them all.

How to answer. Where an exercise asks you to run ESBMC, report the command, the verdict, and — when the verdict is **FAILED** — the counterexample values that make the property fail. A verdict on its own is not an answer; the interesting part is always *why*.

Runtimes. Most runs finish in under a second. The concurrent ones in Sections 4 and 6 take from a few seconds to a few minutes and grow sharply with `--context-bound` and `--unwind`; raise them one step at a time.

1 Bounded model checking

Exercise 1 (Bounded Model Checking)

Describe the steps a bounded model checker takes, from reading the program to producing an SMT formula, using the program below (see [1] for the encoding ESBMC actually uses). The `__ESBMC_assume` directive constrains non-deterministic variables; `assert` states the property to check.

```
#include <assert.h>
#define a 2
int main(int argc, char **argv) {
    long long int i = 1, sn = 0;
    unsigned int n = 5;
    __ESBMC_assume(n >= 1);
    while (i <= n) {
        sn = sn + a;
        i++;
    }
    assert(sn == n * a);
}
```

- (a) Unroll the loop by hand for $n = 5$ and write the resulting single-assignment (SSA) form.
- (b) Split the SSA form into the constraint C and the property P , and give the formula the solver is asked to decide. Which answer — *satisfiable* or *unsatisfiable* — corresponds to VERIFICATION FAILED?
- (c) Run `esbmc bmc-example.c`. Compare the number of assignments reported by *Symex completed* with the number that survives *Slicing*. What has the slicer removed, and why is it sound to remove it?
- (d) Replace `unsigned int n = 5;` with `unsigned int n = nondet_uint();`. The verdict changes. Explain what `--unwind` now controls, and what an *unwinding assertion* is telling you when it fails.

Exercise 2 (Completeness of BMC)

BMC is a bug-finding technique first and a proof technique second.

- (a) State the condition under which VERIFICATION SUCCESSFUL from a bounded run is a *proof* that the property holds for all executions.
- (b) Run `ex1.cpp` at `--unwind 5` twice: once with `--no-unwinding-assertions` and once without. Both report VERIFICATION FAILED, but they fail on different properties — name them. Say which of the two runs you would be willing to put in a safety argument, and note that “FAILED” on its own told you almost nothing here.
- (c) The Python program `collatz` below terminates for `collatz(6)` but not for `collatz(-1)`. Explain what a bounded model checker can and cannot tell you about it.

```
def collatz(n: int) -> int:
    while n != 1:
        if n % 2 == 0:
            n = n // 2
        else:
            n = 3 * n + 1
    return 1
```

2 Loops: invariants, k-induction and abstract domains

Exercise 3 (Strengthening a loop invariant)

`examples/incomplete.c` carries two loop invariants that are deliberately too weak:

```
__ESBMC_loop_invariant(i >= 0);
__ESBMC_loop_invariant(j >= i);
while (i <= n) { i = (i + 1); j = (j + i); }
assert((i + (j + k)) > (2 * n));
```

- (a) Run `esbmc incomplete.c --loop-invariant --no-standard-checks`. Which of the three obligations — *base case*, *inductive step*, *postcondition* — fails, and what does that tell you about where the invariant is too weak?
- (b) Strengthen the annotations until the run reports VERIFICATION SUCCESSFUL. Three things are needed, and the second and third are the interesting ones:
 - i. an upper bound on i that pins its value at loop exit;

- ii. an invariant that fixes j *exactly* rather than bounding it. A range such as $j \geq i$ is inductive but too weak: the inductive step havoc the loop variables, so it admits states the loop can never actually reach. Read the counterexample ESBMC prints and find the havocked state it used;
- iii. bounds on the inputs n and k . Without them the run still fails — read that counterexample too and say which expression overflows.

State the final annotations and argue informally that the invariant is inductive.

- (c) Repeat on the Python version, `examples/incomplete.py`, with `--loop-invariant-check`. The annotation is spelled `__loop_invariant` there. All three of the things above are needed again, including the input bounds — say what that tells you about the integer model ESBMC’s Python frontend uses, and how it differs from CPython’s.
- (d) `examples/crosshair.py` is a solved instance of the same shape. Explain why the equalities $y = i$ and $x = 2i$ are needed and why the bounds $0 \leq i \leq n$ alone would not be enough.

Exercise 4 (k-induction and interval analysis)

- (a) Run `esbmc kinduction-interval.c --k-induction`. The verdict is `VERIFICATION UNKNOWN`. Read the output and say which of the three k-induction steps — base case, forward condition, inductive step — did not go through.
- (b) Add `--interval-analysis` and run it again. It now succeeds. What fact about `status` does the interval domain supply that k-induction could not derive on its own?
- (c) Do the same for `kinduction-interval2.c`, where the loop counter starts non-deterministically.
- (d) `kinduction.c` succeeds with plain `--k-induction`, and the output reports the k at which it closed. Explain what “*Solution found by the inductive step ($k = 2$)*” means operationally, and relate the three steps ESBMC runs — base case, forward condition, inductive step — to the algorithm in [2].

Exercise 5 (Three ways to handle the same loop)

The three Python files `loop-invariant.py`, `loop-invariant2.py` and `loop-invariant3.py` attack one loop in three ways: full unwinding, truncation to one iteration, and replacement by an invariant.

- (a) Run all three (`loop-invariant.py` needs `--unwind 101`) and record the verdicts.
- (b) `loop-invariant2.py` reports `VERIFICATION FAILED` even though the original program is correct. Name the phenomenon this illustrates and say which way it errs — is the tool reporting a bug that cannot happen, or missing one that can? Then say which way a bounded run errs when `--no-unwinding-assertions` is passed. The two answers are opposites, and only one of them is unsoundness.
- (c) `loop-invariant3.py` succeeds without unwinding the loop at all. Which of its `__ESBMC_assume` lines encode the invariant, and which encodes the negated loop condition?
- (d) Which of the three approaches scales to a loop bound of 10^9 , and at what cost to the engineer?

3 Memory safety and arithmetic

Exercise 6 (Memory safety)

- (a) `examples/ex1.cpp` has *two* defects, both on line 7. Find them by inspection, then run `esbmc ex1.cpp --unwind 5 --no-unwinding-assertions`. Only one is reported. Add `--multi-property`, list all four properties ESBMC raises on that line, and say which two hold and why.
- (b) ESBMC distinguishes *array bounds violated*, *NULL pointer*, *invalidated dynamic object* and *incorrect alignment*. Give a one-line C fragment that triggers each, and verify each with ESBMC.
- (c) Fix `ex1.cpp` and show that the fixed version verifies for an unbounded `n`. Plain `--k-induction` returns `VERIFICATION UNKNOWN` here, with or without `--interval-analysis`; a loop invariant relating the counter to `n` does discharge it. State the invariant and the flags you used.

Exercise 7 (Arithmetic and the number model)

- (a) Run `esbmc add-overflow.py`. It succeeds. Now add `--overflow-check`. Explain, in terms of verification conditions, why the first run could not have found the bug.
- (b) Run `esbmc neural-net.c` and then `esbmc neural-net.c --fixedbv`. The same assertion passes under one number model and fails under the other. Explain which model each flag selects and which of the two verdicts describes the real hardware.
- (c) `neural-net.py` computes the same network and succeeds. What does that tell you about comparing verification results across language frontends? See [4] for how the Python frontend is built.

4 Concurrency

Exercise 8 (Interleavings and data races)

- (a) `examples/interleavings.c` runs `x++` twice against `x--` twice with no synchronisation. Enumerate the values `x` can hold once both threads have run to completion, and justify each with an interleaving of the load/add/store steps. Note first that `main` never joins the threads, so the program as written has no such point — say what that means for the question, and add the two `pthread_join` calls before answering it.
- (b) Run `esbmc interleavings.c --data-races-check`. Note that ESBMC reports the race without any assertion in the program. What is the difference between a *data race* and an *assertion violation*?
- (c) Read the *Schedule reduction* line at the end of the output. Say what `peak_threads`, `schedules_explored` and the four `pruned_by_*` counters report. On this example every pruning counter is *zero* — explain why partial-order reduction finds nothing to prune here, and construct a variant of the program on which `pruned_by_mpor` is non-zero.

Exercise 9 (Atomicity violation in a shared bank account)

`examples/bank-account-bug.c` checks the balance and then withdraws, with the two steps outside any lock. `examples/bank-account.c` is the same code with a mutex around both.

- (a) Run both with `--unwind 2 --no-unwinding-assertions`. Report the two verdicts and, for the failing one, the counterexample values of `balance`.
- (b) The bug is not a data race in the strict sense — each individual access could be atomic and the bug would remain. Explain why, and name the class of defect this is.

- (c) Suppose the `pthread_mutex_unlock` in `bank-account.c` were moved to just after the `if (balance >= amount)` test. Explain what goes wrong, and give the interleaving of the two threads that overdraws the account. Then explain why this variant is far more expensive for ESBMC to check than either of the two originals — it does not finish within a few minutes at these bounds — in terms of how many context switches the split critical section admits.

Exercise 10 (Mutual exclusion)

Consider Peterson’s algorithm as implemented in `examples/mutual-exclusion.c`.

```
void *t1(void *arg) {
    flag[0] = 1;
    turn = 1;
    while (flag[1]==1 && turn==1) {};
    c1 = 1;
    CT0: if (i==1) x=1;
    c1 = 0;
    flag[0] = 0;
    return NULL;
}

void *t2(void *arg) {
    flag[1] = 1;
    turn = 0;
    while (flag[0]==1 && turn==0) {};
    c2 = 1;
    CT1: if (i==2) x=2;
    c2 = 0;
    flag[1] = 0;
    return NULL;
}
```

- (a) Specify the following three properties in LTL, using c_i for “process i is in its critical section” and r_i for “process i has requested entry” ($i \in \{1, 2\}$; the propositions are named apart from the functions `t1` and `t2` on purpose).
- At most one process is in the critical region at any time.
 - Whenever a process tries to enter its critical region, it will eventually succeed.
 - A process can eventually ask to enter its critical region.
- (b) Which of the three are *safety* properties and which are *liveness* properties (see [6])? Which of them can a bounded run refute, and which can it only fail to refute?
- (c) Verify property (i):

```
esbmc mutual-exclusion.c --unwind 3 --context-bound 3 \
    --no-unwinding-assertions
```

- (d) Show that the critical section is reachable at all, by checking the label `CT0` with `--error-label CT0`. Why is a bare `--error-label` verdict *not* on its own evidence that a label is unreachable?
- (e) Delete `turn = 1;` from `t1` and re-verify. Report the interleaving ESBMC produces.

Exercise 11 (Locking a counter)

`examples/pthread.c` has two threads increment a shared `n` ten times each under a mutex, and asserts `n == 20`.

- (a) Verify it with `--unwind 11 --context-bound 2`.
- (b) Remove the `pthread_mutex_lock/unlock` pair and re-verify at the same bounds (`--unwind 11 --context-bound 2`). What is the smallest final value of `n` that ESBMC can justify with an interleaving? The answer depends on the context bound — say what it would become without one.
- (c) Raise `--context-bound` and observe the runtime. State precisely what a run with `--context-bound k` does *not* prove.

5 Temporal logic

Exercise 12 (Specifying and checking an LTL property)

ESBMC checks LTL by compiling the *positive* formula into a Büchi monitor with the ESBMC fork of `ltl2ba` and running it as an extra thread. The tool performs the negation internally.

```
ltl2ba -O c -H '"vars.h"' -f 'G({pressed} -> F {charge > min})' \
    > ltl_example.ba-2.c
esbmc ltl_example.c --ltl ltl_example.ba-2.c -DLTL_PREFIX_BOUND=10
```

- Run the two commands above on `examples/ltl_example.c` and report the *Final lowest outcome*. ESBMC distinguishes `LTL_BAD`, `LTL_FAILING` and `LTL_SUCCEEDING` — explain what each means for a *finite prefix* of an infinite trace, using the good-prefix/bad-prefix vocabulary of [6].
- Explain the role of `LTL_PREFIX_BOUND`. Reduce it to 1 and explain the resulting verdict.
- Reduce the loop in `ltl_example.c` to a single iteration. The verdict becomes `VERIFICATION UNKNOWN`. Why does the monitor need more than one step to reach a decision?
- Write and check the LTL formula $G(\text{pressed} \rightarrow X(\text{charge} > \text{min}))$. Is it stronger or weaker than the formula above? Confirm your answer with ESBMC.

Exercise 13 (Specifying a chemical-process controller)

A typical embedded system consists of a human–machine interface (keyboard and display), a processing unit, and an instrumentation interface (sensors, network, actuators) connected to a physical plant. The system considered here controls a chemical process and can be operated remotely over TCP/IP. It has two ADCs — one reading thermocouples, one reading a pressure transducer — and a DAC that sets the valve/pump position, plus a switch for the heater. Its objective is to keep temperature and pressure within defined limits.

- Specify each of the following in LTL:
 - A T process reads a temperature sensor, turns the heating system on if the temperature is below 20 °C, and turns it off if the temperature is above 300 °C.
 - A P process regulates pressure, opening the pump/valve when pressure is above 500 bar and closing it when pressure is below 100 bar.
 - The measured values are shown on an LCD when the T and P processes transfer data to an S process.
- Which of your three formulas are safety properties? For each safety property, give the corresponding C assertion.
- What are the security objectives of this system?
- What are the sources of security problems that arise when it is operated remotely? Consider the TCP interface specifically.

6 Verifying LLM-generated code

Exercise 14 (Reviewing an LLM implementation with a model checker)

The directory `ai-generated/` contains three LLM-written implementations of the controller from Exercise 13: `embedded_system_llm.c` (no synchronisation), `embedded_system_unsafe1.c` (one global mutex) and `embedded_system_unsafe2.c` (one mutex per shared variable).

```

esbmc embedded_system_llm.c      --data-races-check --context-bound 2 \
      --unwind 2 --no-unwinding-assertions
esbmc embedded_system_unsafe2.c --data-races-check --context-bound 2 \
      --unwind 2 --no-unwinding-assertions
esbmc embedded_system_unsafe1.c --data-races-check --context-bound 1 \
      --unwind 2 --no-unwinding-assertions

```

The first two take one to two minutes; the third takes a few seconds. Note the lower context bound on the third — at `--context-bound 2` it does not finish in several minutes.

- (a) Run all three. Report the verdict for each and, where one fails, the first property violated.
- (b) The third file reports **VERIFICATION SUCCESSFUL**, yet it takes the same non-recursive mutex twice on the path `temperature_process` $\rightarrow$ `set_heater`, which self-deadlocks under POSIX. Adding `--deadlock-check` does not change the verdict either. Explain why the bounded run cannot see this defect — look at what the worker threads do, and at where ESBMC places its deadlock property — and propose a change to the harness that would expose it.
- (c) In `embedded_system_unsafe2.c` the assertion relating `temperature` to `heater_status` was moved into the display process, and each variable has its own lock. Explain why per-variable locking does not preserve the relation between them.
- (d) All three files were produced by a language model from the same natural language specification, and all three look plausible. Write two or three sentences on what this implies for reviewing generated code, and on what a model checker adds that a human reviewer does not. Reference [5] describes one way to close the loop: feed the counterexample back to the model and re-verify the repair.
- (e) Repair one of the three so that it verifies, and state the property your repaired version establishes and the bounds under which it establishes it.

Exercise 15 (Writing a specification an LLM cannot fake)

`examples/llm_thermostat_example.c` is a short LLM-written guard that ESBMC reports as **VERIFICATION SUCCESSFUL**.

- (a) Read the program and explain why the successful verdict is close to meaningless.
- (b) Rewrite it so that the temperature is non-deterministic and the assertion states a property of the *controller* rather than of a constant. Verify your version.
- (c) Formulate one property of a thermostat that BMC cannot establish at all, and say what technique you would use instead.

References

- [1] L. C. Cordeiro, B. Fischer, J. Marques-Silva. “SMT-Based Bounded Model Checking for Embedded ANSI-C Software”. *IEEE Transactions on Software Engineering* 38(4):957–974, 2012. doi:10.1109/TSE.2011.59
- [2] M. Y. R. Gadelha, H. I. Ismail, L. C. Cordeiro. “Handling loops in bounded model checking of C programs via k-induction”. *International Journal on Software Tools for Technology Transfer* 19(1):97–114, 2017. doi:10.1007/s10009-015-0407-9
- [3] M. Y. R. Gadelha, F. R. Monteiro, J. Morse, L. C. Cordeiro, B. Fischer, D. A. Nicole. “ESBMC 5.0: an industrial-strength C model checker”. *ASE 2018*, pp. 888–891. doi:10.1145/3238147.3240481

- [4] B. Farias, R. Menezes, E. B. de Lima Filho, Y. Sun, L. C. Cordeiro. “ESBMC-Python: A Bounded Model Checker for Python Programs”. *ISSTA 2024*, pp. 1836–1840. doi:10.1145/3650212.3685304
- [5] N. Tihanyi, Y. Charalambous, R. Jain, M. A. Ferrag, L. C. Cordeiro. “A New Era in Software Security: Towards Self-Healing Software via Large Language Models and Formal Verification”. *AST@ICSE 2025*, pp. 136–147. doi:10.1109/AST66626.2025.00020
- [6] C. Baier, J.-P. Katoen. *Principles of Model Checking*. MIT Press, 2008. ISBN 978-0-262-02649-9.
- [7] A. Burns, A. J. Wellings. *Real-Time Systems and Programming Languages: Ada, Real-Time Java and C/Real-Time POSIX*, 4th edition. Addison-Wesley, 2009. ISBN 978-0-321-41745-9. Source of the chemical-process case study in Exercise 13.